Politecnico di Torino

Testing and Fault Tolerance

Software-Based Self-Test (SBST) Development for the BPU of the CORE-V Wally Processor

Project Report and Manual

Master Degree in Computer Engineering

Baraldi Andrea (s339846) - Marocco Luca (s333945) - Mugnaini Andrea (s338780)

March 7, 2026

Contents

Introduction

This report documents the development and evaluation of a Software-Based Self-Test suite targeting the **Branch Prediction Unit** (BPU) of the CORE-V Wally (CVW) processor. The objective was to achieve high fault coverage (FC) for Stuck-At Faults (at least 90%) and Transition Delay Faults (at least 80%) while minimizing the Test Application Time (TAT).

What SBSTs do

SBSTs target **faults introduced during the physical manufacturing process:** a gate output permanently stuck at logic 0 or 1 (SAF) or a signal that fails to transition within the required clock period (TDF) can exist in a chip that passes all functional tests and these programs can detect these physical defects on the processor itself, without requiring external test equipment.

The CVW Environment

The CORE-V Wally is a configurable RISC-V processor that implements a **5-stage pipeline**: Fetch (F), Decode (D), Execute (E), Memory (M) and Writeback (W).

The **Instruction Fetch Unit** (IFU) spans the first stage and is responsible for supplying a continuous stream of instructions to the pipeline. It comprises the PC and Branch Prediction logic, an Instruction MMU, an optional Instruction Cache and a bus interface to external memory.

Our target module, the BPU, is responsible for **minimizing control flow stalls** by predicting the direction (Taken/Not Taken) and target address of branch and jump instructions during the Fetch (F) stage (code located at `src/ifu/bpred/bpred.sv`).

Configuration and Optimization

As a customizable plaftorm, the CVW environment introduces the concept of **derivatives**, specific architectural configurations of the core, defined by a set of parameters (ISA extensions, cache sizes, etc.) located in the `config/` directory. The hardware configuration is controlled through the `config/derivlist.txt` file.

The full derivative that we used is called `syn_polito_rv32e_bpu` and the configuration parameters are shown in Listing A.1. Beyond the BPU-specific parameters, which we talk about later in the report, the configuration enables the full RISC-V base integer instruction set with 32 registers **(`E_SUPPORTED = 0`, despite the name)** and activates both the base counter infrastructure (`ZICNTR_SUPPORTED = 1`) and the hardware performance monitor extension (`ZIHPM_SUPPORTED = 1`) with 32 counters (`COUNTERS = 12'd32`). The HPM counters in particular are essential, as they serve as an observability mechanism for fault detection in the absence of direct access to internal BPU signals.

CHAPTER 1

The Branch Prediction Unit

1.1 Architectural Analysis

The **BPU**, implemented in the gate-level netlist hierarchy inside the IFU, is actually a wrapper class for 3 subcomponents:

- **BHT** - Branch History Table
- **BTB** - Branch Target Buffer
- **RAS** - Return Address Stack

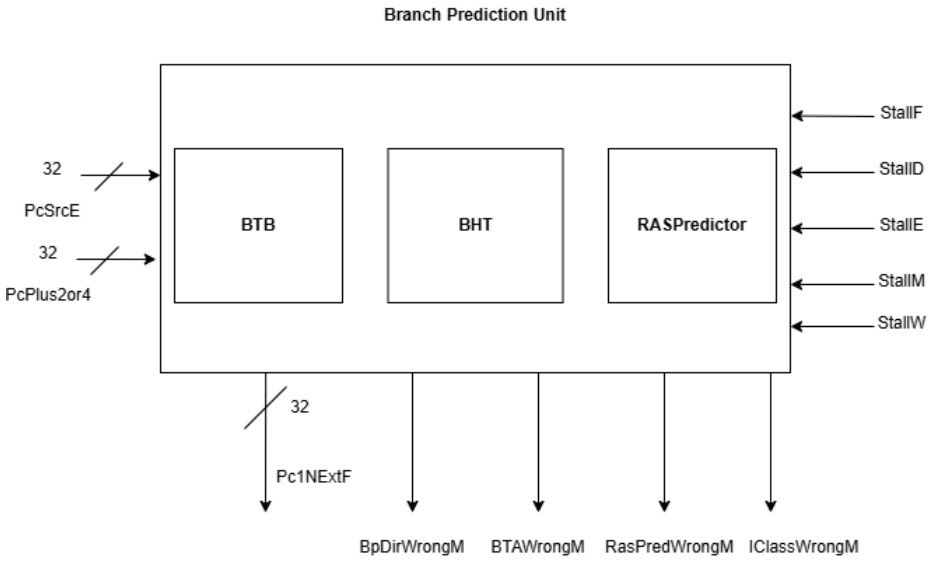


Figure 1.1: BPU

In terms of derivative parameters, we set `BPRED_SUPPORTED` at `1`, which enables the BPU.

1.1.1 Branch History Table (BHT)

In our configuration, the BHT is a direct-mapped table of 8 entries (`BPRED_SIZE = 3`, so $2^3 = 8$), where each entry is a **2-bit saturating counter** (`BPRED_TYPE = 'BP_TWOBIT`) that predicts the direction of a branch (Taken or Not Taken) based on its recent history.

Each counter implements a **four-state FSM** with states Strongly Not Taken (00), Weakly Not Taken (01), Weakly Taken (10), and Strongly Taken (11), where the MSB determines the prediction: 0 means predict Not Taken, 1 means predict Taken. The counter saturates at both ends, meaning

that a branch must misbehave twice in a row before the prediction flips, which makes the predictor able to overcome isolated anomalies in loop behavior.

The BHT is indexed by the lower bits of the Program Counter (`PCNextF`), so each instruction address maps to a specific BHT entry, and because the table has only 8 entries, multiple branches can alias to the same slot. It operates across two pipeline stages: in the Fetch stage, the counter is read to produce the prediction (`BPDirF`), and in the Memory stage, after the branch outcome is known, it is updated with the actual direction (`NewBPDirM`). Despite more complex configurations being available (e.g. GShare), the 2-bit Bimodal predictor was chosen for its simpler structure, which makes it more susceptible to exhaustive march-based testing.

1.1.2 Branch Target Buffer (BTB)

The BTB is a direct-mapped cache of 8 entries (`BTB_SIZE = 3`, so $2^3 = 8$) that stores two pieces of information for each control flow instruction: the target address it should jump to and the instruction class (a 4-bit encoding identifying whether the instruction is a branch, jump, JAL or return).

It is indexed using a **hashed version** of the **PC**, to improve address distribution across entries. Like the BHT, it is updated only on mispredictions to conserve power: when the predicted target address (`BPBTAE`) differs from the actual computed target (`IEUAdrE`) in the Execute stage, or when the predicted instruction class is wrong, the `BTBWrongM` signal triggers a write to the BTB table in the Memory stage.

Although direct branch and jump targets (JAL, branches) are relative to the PC and never change, the BTB predicts them with accuracy once it has learned them. Indirect branches (JALR, ret), whose target depends on a register value, are harder to predict and are where the BTB most frequently mispredicts, motivating a dedicated RAS mechanism.

1.1.3 Return Address Stack (RAS)

The RAS is a hardware **stack structure** dedicated to predicting return addresses, a task the BTB handles poorly, as said before. It has up to 10 entries (`RAS_SIZE = 32'd10`).

The stack operates across two pipeline stages: in the Execute stage, when a call instruction is detected (`CallE`), the return address (`PCLinkE` = PC + 4) is pushed onto the stack at position `NextPtr` and the pointer is incremented, in the Fetch stage, when an instruction is predicted to be a return (`BPReturnF`), the top of the stack (`RASPCF`) is popped and used directly as the predicted target address, decrementing the pointer.

Since the BTB predicts instruction class speculatively, before the instruction reaches Decode, it can misclassify instructions and leave the RAS pointer in an inconsistent state. Two repair signals correct this: `IncrRepairD` increments the pointer when a non-return was wrongly predicted as a return (undoing a spurious pop), `DecRepairD` decrements it when a return was missed (compensating for a pop that never happened).

CHAPTER 2

SBSTs

Each test is implemented as a **standalone assembly routine**, called sequentially from a C `main()` function that handles signature collection and comparison.

2.0.1 Signature-verification

Prediction correctness is observed through five **Hardware Performance Monitor** (HPM) counters, sampled via CSR reads (`0xB04`–`0xB09`), and corresponding to Jump Not Return, Return, BP Wrong, BP Dir Wrong, and BP Target Wrong events respectively. The signature for each test is computed as the **difference** between a snapshot taken immediately before and immediately after the test routine executes, isolating only the BPU events attributable to that test. While not all counters are relevant to every test, the same five-counter delta is computed uniformly across all tests for simplicity, and the resulting 32-bit value is compared against a hardcoded **golden signature** to determine pass or fail.

Listing 2.1: HPM Counters

```
Cnt[ 0] =   21976 Mcycle
Cnt[ 1] =       0 ------
Cnt[ 2] =    4342 InstRet
Cnt[ 3] =     163 Br Count
Cnt[ 4] =     515 Jump Not Return
Cnt[ 5] =     670 Return
Cnt[ 6] =    1816 BP Wrong
Cnt[ 7] =      55 BP Dir Wrong
Cnt[ 8] =     580 BP Target Wrong
Cnt[ 9] =     388 RAS Wrong
Cnt[10] =    1848 Instr Class Wrong
Cnt[11] =     196 Load Stall
Cnt[12] =      64 Store Stall
Cnt[13] =       0 D Cache Access
Cnt[14] =       0 D Cache Miss
Cnt[15] =       0 D Cache Cycles
Cnt[16] =       0 I Cache Access
Cnt[17] =       0 I Cache Miss
Cnt[18] =       0 I Cache Cycles
Cnt[19] =       3 CSR Write
Cnt[20] =       0 FenceI
Cnt[21] =       0 SFenceVMA
Cnt[22] =       0 Interrupt
Cnt[23] =       0 Exception
Cnt[24] =       0 Divide Cycles
```

2.1 Test 1

The first test is designed to stress the **BHT** by exercising all saturating counter states and transitions, targeting the `BPDirWrongE` misprediction signal as the primary observable output.

2.1.1 Methodology

Following the approach of Sanchez et al. [1], the test implements an adapted **memory march algorithm** for BHT saturating counters. The key challenge in stimulating a BHT is that its entries can only be read and written indirectly, through the execution of branch instructions whose addresses hash to the desired BHT entry. Two design decisions follow from this: first, **procedure calls** are used rather than inline branches, so that the call overhead instructions do not alias to the same BHT entries under test and corrupt the counter state and second, the linker script is modified to place each stub function at a precisely computed address, ensuring that the `beq` instruction inside each stub maps to a distinct BHT entry (see Section 2.1.3).

Each stub has the following structure: a single `beq x10, x12` instruction followed by a `nop` and a `ret`. Branch direction is controlled globally by loading either `A = 0` or `B = 1` into `x12`: when `x10 == x12` the branch is Taken, otherwise Not Taken. This allows the same eight stubs to serve all three march phases simply by changing one register value.

2.1.2 March Test Interpretation

The test is composed of the following march elements:

$$M_1 :\uparrow (w11) \quad M_2 :\downarrow (r11, w00) \quad M_3 :\uparrow (r00, w11)$$

Phase 1 – Initialization (March M_1)

All 8 BHT entries are saturated to Strongly Taken (11) by calling each stub 3 times with `x12 = 0` (Taken). Three taken branches are sufficient to reach state 11 from any initial state.

$$M_1^{BHT} :\uparrow (w(11))$$

Phase 2 – Downward Scan (March M_2)

With `x12 = 1` (Not Taken), each stub is called 4 times in **descending** order (entry 7 down to 0). The first two calls generate mispredictions (11→10→01, predicting Taken but seeing Not Taken), and the last two are correct predictions (01→00), fully exercising all downward transitions.

$$M_2^{BHT} :\downarrow (r(11), w(00))$$

Phase 3 – Upward Scan (March M_3)

With `x12 = 0` (Taken), each stub is called 4 times in **ascending** order (entry 0 up to 7). Symmetrically to Phase 2, the first two calls produce mispredictions (→01→10) and the last two are correct (10→11), covering all upward transitions.

$$M_3^{BHT} :\uparrow (r(00), w(11))$$

2.1.3 Placing calls to a specific address

Because the BHT is indexed by a hash of the PC, placing branch instructions at arbitrary addresses does not guarantee that all 8 entries are reached. The BHT hashing function was reverse-engineered and a Python script was used to compute, for each of the 8 entries, a set of addresses that map to it. The linker script was then modified to force each stub function to one of these addresses using explicit section placement directives, as shown in Listing A.2.

Each address is spaced 0x44 bytes apart, which corresponds to the stride required by the Wally hash function to advance by one BHT index. This ensures that all 8 entries are independently targeted without aliasing between stubs.

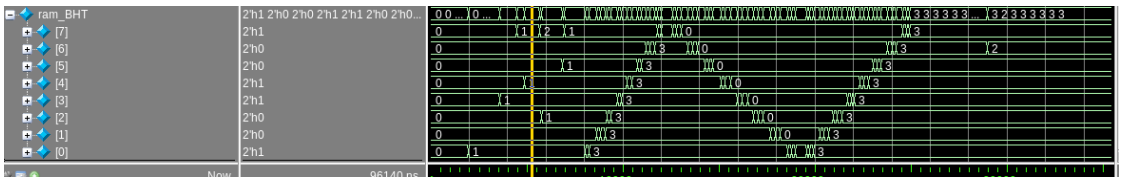


Figure 2.1: BHT saturating counters march test

This guarantees full **static coverage** of the address decoding logic, all 2-bit counter states, all state transitions and prediction and update logic.

2.1.4 Scaling

The BHT has can be reverse-engineered for BHT entries of any size. Each BHT entries is indexed by the following hash, present in `twobitspredictor.sv`:

Listing 2.2: Hardware indexing logic for the Branch History Table.

```
// hashing function for indexing the PC
// We have k bits to index, but XLEN bits as the input.
// bit 0 is always 0, bit 1 is 0 if using 4 byte instructions, but is not always 0
    if using compressed instructions.  XOR bit 1 with the MSB of index.
assign IndexNextF = {PCNextF[k+1] ^ PCNextF[1], PCNextF[k:2]};
assign IndexM = {PCM[k+1] ^ PCM[1], PCM[k:2]};
```

Let K be the number of bits required to address a BHT of size 2^K, the hardware calculates the K-bit index, denoted as $IndexM$.

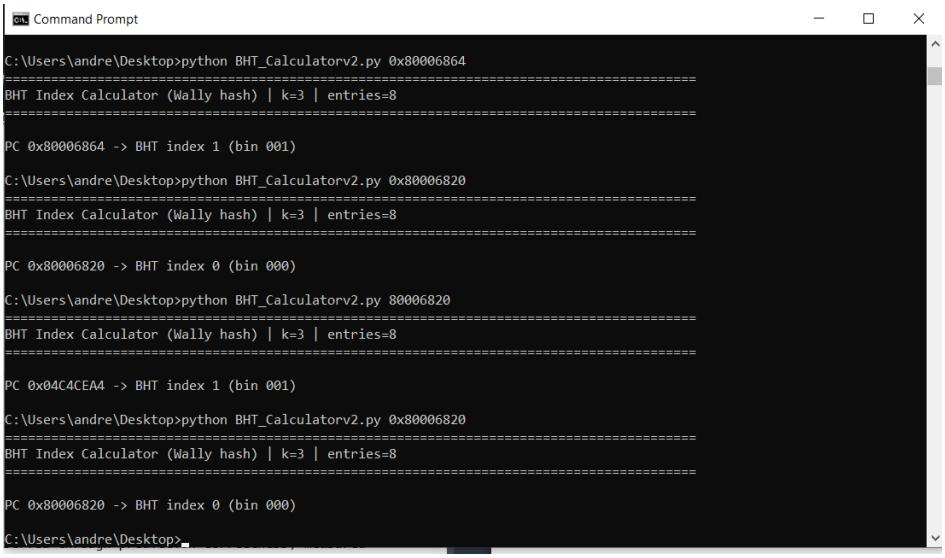


Figure 2.2: Python Script to find the correct addresses to stimulate BHT entries

To test specific patterns, such as all 0s or 1s on the two-bit bit predictor requires to issue a jump instruction(or a branch, call) as many times as the amount we want to force inside the specific BHT entry, indexed by the branch address location.

2.2 Test 2

The second test, located in `sbst2.S`, stresses the **BTB**.

2.2.1 Methodology

To accurately stimulate and observe the BTB, we analyzed a single entry within the `btb.sv` module, which is exactly 36 bits wide.
Following the approach of Changdao et al. [5], the test requires a set of patterns (ideally $w0$ and $w1$) to stress the memory via a **modified MATS+ March Algorithm**. However, the 32-bit `IEUAdr` target field constrains the logical patterns available for memory testing, and, because it is injected directly into the fetch pipeline as the `NextPC`, any sequence designed to test the 36-bit entry must consist of valid, executable addresses.
For this reason, the two logical patterns were chosen at opposite extremes of the available address space. The **Logic 0** pattern was set to **0x80002280**, the first correctly aligned address from the base of the RAM as defined in the linker script. Symmetrically, the **Logic 1** pattern was set to **0x87FFFE04**, a value close to the highest address available in our implementation. This maximizes the Hamming distance between the two patterns, improving the ability to detect both stuck-at and transition faults across all 32 bits of the target field.

2.2.2 Architectural Mapping of the MATS+ Algorithm

This 36-bit word is formed by the concatenation of two distinct fields:

- `IClass` **(4 bits):** Located at bits `[35:32]`, this field stores the Branch Predictor's guess regarding the instruction class. The hardware uses these 4 bits to identify whether the control flow instruction is a `jalr`, `return`, `jump`, or `branch`.
- `IEUAdr / BPBTA` **(32 bits):** Located at bits `[31:0]`, this field stores the Branch Predictor Branch Target Address. If the BTB detects a match during the Fetch stage, it extracts this 32-bit address and speculatively uses it as the Program Counter (PC) for the next fetch cycle.

When the BTB is updated during the Execute or Memory stages following a branch misprediction, the hardware writes the corrected 4-bit `IClass` and the 32-bit resolved target address back into this exact 36-bit memory slot.

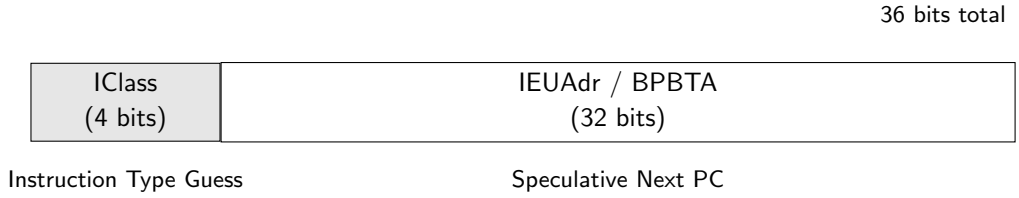


Figure 2.3: Architectural fields of a 36-bit BTB Entry.

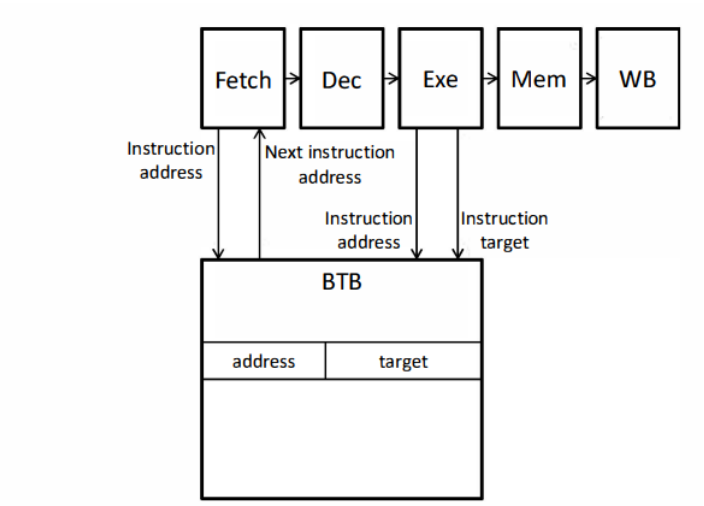


Fig. 1. Interaction of a BTB with the stages of a pipelined processor

2.2.3 Mapping MATS+ to the SBST Implementation

Table 2.1 summarizes how the MATS+ memory test algorithm is physically mapped to the BTB. The logical read and write operations are achieved by executing branches from specific memory sections, forcing the BTB to read its current prediction and subsequently overwrite it with a new target address.

March Element	Memory Section	Read Entry (Fetch Stage)	Target Written (Execute Stage)
$\Uparrow (w0)$	`.w0_entry0`	Previous value	Logical 0 (`0x80002280`)
$\Uparrow (r0, w1)$	`.w1_entry0`	Logical 0 (`0x80002280`)	Logical 1 (`0x87FFFE04`)
$\Downarrow (r1, w0)$	`.w0d_entry0`	Logical 1 (`0x87FFFE04`)	Logical 0 (`0x80002280`)

Table 2.1: Simplified mapping of the MATS+ algorithm, detailing the memory sections and address transitions used to functionally test the BTB.

When our SBST algorithm executes the MATS+ elements (such as the transition $\uparrow (r0, w1)$) it uses branch instructions where the destination address evaluates to the high address (0x87FFFE04) memory block. By forcing a misprediction, the processor computes the new `IEUAdr` and writes it into the low 32 bits of the entry, while simultaneously evaluating the instruction type and writing the corresponding 4-bit `IClass` into the upper bits.

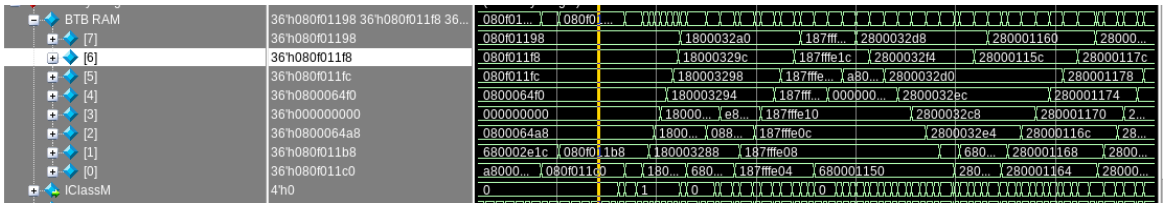


Figure 2.4: MATS+ March implementation

Hardware Flow of $\uparrow (r0, w1)$

When the test program executes the transition element $\uparrow$ $(r0, w1)$ by jumping to the `W1_SITE0_ascending` routine, the hardware sequence follows this flow:

1. **Cycle T (Read 0):** The Fetch stage reads the BTB at index `000`. The SRAM (`ra1`) outputs the previously stored logical 0 target (e.g., `0x80002280`).
2. **Cycle $T+1$ (Execute):** The processor calculates the actual branch target for the `w1` instruction, which is `0x87FFFE04`. It detects a mismatch against the predicted `0x80002280`, asserting the misprediction signal.
3. **Cycle $T+2$ (Write 1):** In the Memory stage, `BTBWrongM` goes high, asserting `we2`. The SRAM physically writes the new logical 1 data (`IClassM` and `0x87FFFE04`) into index `000` (`wa2`).

By sequentially executing the branch stubs mapped via the linker script, the pipeline acts as an automated memory built-in self-test (MBIST) engine, rhythmically pulsing the SRAM's read and write ports to guarantee transition fault and stuck-at fault coverage across the BTB.

2.2.4 MATS+ Scaling

The proposed SBST methodology can be scaled to Branch Target Buffers of varying depths (e.g., 8, 16, 32, or more entries). BTB addressing uses the $[n+1:2]$ bits, for an N-entry BTB where $N = 2^n$) is used to index the SRAM array. **Scaln**

Low Memory Mapping (Logical "0")

To initialize the array with logical 0s, the branch stubs are mapped to a low memory base. Listing A.3 demonstrates the linker configuration for the ascending ($\uparrow (w0)$) and descending ($\downarrow (r1, w0)$) sections. The use of `ALIGN(64)` is required, since it forces the base address to end in binary `000000`, so that we are able to address the first entry. For the descending traversal (`.w0d`), the linker script requires using `ALIGN(8)` for even entries and explicitly advancing the location counter (`. += 4`) before the odd entries.

High Memory Mapping (Logical "1")

To execute the transition element $\uparrow (r0, w1)$, the test scales into a higher memory region, shown in Listing A.4.

```
Disassembly of section .w0d_entry0:

80002080 <W0_SITE0_descending>:
80002080:   0040006f            j  80002084 <W0_SITE0_descending+0x4>
80002084:   00008067            ret

Disassembly of section .w0d_entry2:

80002088 <W0_SITE2_descending>:
80002088:   01c0006f            j  800020a4 <W0_SITE1_descending>
8000208c:   00000013            nop

Disassembly of section .w0d_entry4:

80002090 <W0_SITE4_descending>:
80002090:   01c0006f            j  800020ac <W0_SITE3_descending>
80002094:   00000013            nop

Disassembly of section .w0d_entry6:

80002098 <W0_SITE6_descending>:
80002098:   01c0006f            j  800020b4 <W0_SITE5_descending>
8000209c:   00000013            nop

Disassembly of section .w0d_entry1:

800020a4 <W0_SITE1_descending>:
800020a4:   fddff06f            j  80002080 <W0_SITE0_descending>
800020a8:   00000013            nop

Disassembly of section .w0d_entry3:

800020ac <W0_SITE3_descending>:
800020ac:   fddff06f            j  80002088 <W0_SITE2_descending>
800020b0:   00000013            nop

Disassembly of section .w0d_entry5:

800020b4 <W0_SITE5_descending>:
800020b4:   fddff06f            j  80002090 <W0_SITE4_descending>
800020b8:   00000013            nop

Disassembly of section .w0d_entry7:

800020bc <W0_SITE7_descending>:
800020bc:   fddff06f            j  80002098 <W0_SITE6_descending>
800020c0:   00000013            nop
```

Figure 2.5: Memory dump of the ascending write-1 section of the march test

To scale this algorithm,we simply need to increase `.wX_entry` definitions in the assembly and linker scripts up to N.

2.2.5 BTB Stress tests

Apart from the MATS+ memory testing, the SBST routine executes a series of targeted functional tests to stress the control logic, boundary conditions, and fetch mechanisms of the Branch Prediction Unit (BPU).

1. **BTB Thrash Test** Executes 64 consecutive jumps to intentionally overflow the Branch Target Buffer, verifying its control logic and eviction policies for the entries.

2. **Condition Coverage:** Sequentially evaluates all RISC-V conditional branches (`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`) to be sure the faults didn't affect the BPU condition evaluation logic.

3. **Dense Branching:** Sequential. rapid calls to stress stall and pipeline flushes.

4. **Recursive calls:** Executing deep recursive call and return chains stress the `IClass` field of the BTB, to identify call-return instructions mismatch.

5. **Indirect Target Switch:** The test trains the indirect branch predictor by repeatedly jumping to the same address, then intentionally swaps the target register's value on the final iteration to force a misprediction.

2.3 Test 3

Test 3, located in `sbst3.S`, is designed to stress the RAS (10 entries, `RAS_SIZE = 10`) , targeting its pointer logic, push/pop control signals and wraparound behavior (stress tests).

2.3.1 Methodology

The test is structured in **seven phases**, each targeting a distinct aspect of the RAS behavior. After each phase, snapshots of eight HPM counters (`cycle`, `time` and six event counters) are XORed into `s11`, o any fault that causes execution to visit the wrong address, skip a landing pad or corrupt the sequence produces a final signature that diverges from the golden value.

Phase 1 – 10-deep call chain (lower address range)

A 10-level nested call chain is placed in the `.ras_walk_t3` section, linked to the `0x80008xxx` address range (Listing A.5). Each site saves `ra` on the stack, calls the next site via `jal`, then on return restores `ra` and adds a unique prime to `s11` before executing `ret` (the primes used are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, one per level). The last site calls a `ret` leaf (`chain0_leaf`), which fills all 10 RAS entries with distinct return addresses in the `0x80008xxx` range, then the chain unwinds through all 10 landing pads.

Phase 2 – 10-deep call chain (upper address range)

An identical chain is placed in the `.ras_fill_t3` section (Listing A.5), producing return addresses in a different address range. The same prime sequence (2, 3, 5, 7, 11, 13, 17, 19, 23, 29) is accumulated, providing a second independent check that the full bit-width of each RAS entry is exercised.
Following this phase, `mscratch` is written and read back, with the result XORed into `s11` to additionally exercise CSR datapaths.

Phase 3 – RAS overflow and pointer wraparound

An 11-level nested call chain (`p7_push11_start` through `p7_d11`) is placed in 11 separate linker sections (`.p7_1` through `.p7_11`), ensuring each level's return address lands at a distinct alignment. The 11th `jal` overflows the 10-entry RAS, wrapping the pointer from 9 to 0 and overwriting the oldest entry. The 11 nested `ret` instructions then unwind, with the pointer wrapping from 0 back to 9 at the underflow boundary. Both edges of the pointer are exercised within a single push/pop sequence.

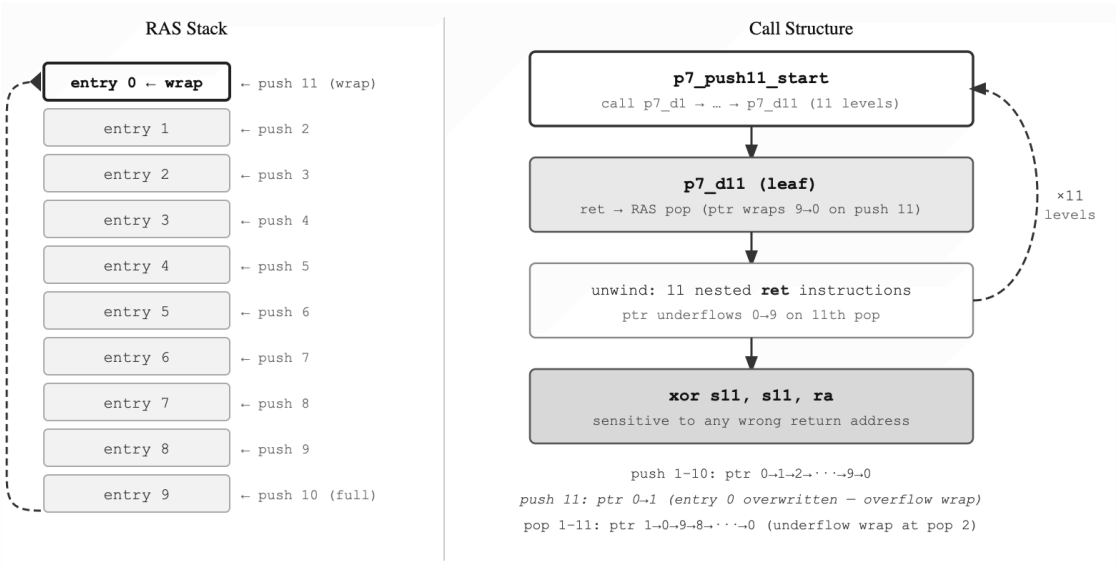


Figure 2.6: RAS Overflow Logic

Phase 4 – JALR rd=x0 non-push verification

Four `jalr x0, t0, 0` instructions jump to distinct targets (`jalr_tgtA` through `jalr_tgtD`). Since `rd=x0`, the RAS must **not** push a return address for any of them. Each landing pad adds a fixed token (0x11, 0x22, 0x33, 0x44 respectively) to `s11`. A subsequent `jal ra, ras_jalr_probe` / `ret` probe then verifies that the RAS top was not corrupted: the probe's return lands at `jalr_probe_land` and adds token `0x77` to `s11`. If a push-enable fault caused any `jalr x0` to incorrectly push, the probe's `ret` pops a garbage entry, misses `jalr_probe_land`, and the token `0x77` is never accumulated.

Phase 5 – Forced mismatch, comparator coverage

Four mismatch pairs exercise the RAS misprediction comparator. Each pair follows the same pattern: a `jal ra, mm_leafN` pushes a predicted return address (the instruction immediately after the `jal`), then `ra` is overwritten with a different destination using `la`, and `ret` is executed. Execution jumps to the `la`-loaded destination (`mm_dstN`) while the RAS had predicted the original landing pad (the mismatch triggers a pipeline flush and toggles the comparator flip-flops). A poison instruction immediately at the predicted (post-`jal`) address adds `0x1FF` to `s11` if it ever executes, which would indicate the RAS prediction was incorrectly committed rather than flushed. Each correct destination adds a distinct token (0x101, 0x102, 0x103, 0x104) to `s11`.

Phase 6 – TDF-F/TDF-R on RAS data bits

Ten alternating push/pop pairs target Transition Delay Faults on the RAS data storage. Each pair calls two stubs (`p6_call_aN` and `p6_call_bN`) placed in separate sections with different alignments (`.align 2` for a-variants and `.align 4` for b-variants), guaranteeing that their return addresses differ in at least bit 2. Calling an a-variant immediately followed by the corresponding b-variant forces the same RAS slot to see a 1→0 transition on at least one bit (TDF-F) and a 0→1 transition on another (TDF-R). After all 10 pairs, every RAS entry has been written with both transition directions. The `ra` value returned from each call is XORed into `s11`, making the signature sensitive to any bit that fails to transition correctly within the clock period.

Phase 7 – Upper-bit SA-0 and TDF-F for bits 16–25

Phase 7 targets Stuck-At-0 and Transition Delay Faults on the upper address bits of the RAS data storage. For each bit $N \in \{16, 17, \ldots, 25\}$, a dedicated 10-deep chain (`hi_bN_n1` through `hi_bN_n10`) is placed by the linker at an address with bit N set (e.g. `0x80010000` for bit 16, `0x80020000` for bit 17, up to `0x82000000` for bit 25), ensuring that all 10 RAS entries are loaded with return addresses where bit $N = 1$. Each node in the chain XORs the returned `ra` into `a0`, which is then folded into `s11` after the chain returns — this detects SA-0 on bit N. Ten distinct low-address leaf functions (`lo_leaf_bN_0` through `lo_leaf_bN_9`) then overwrite all 10 RAS slots with addresses where bit $N = 0$, creating the 1→0 transition that detects TDF-F. The high-address chain is then called a second time to re-confirm SA-0 coverage. This three-step sequence (*high chain → low leaves → high chain*) is repeated independently for all ten target bits.

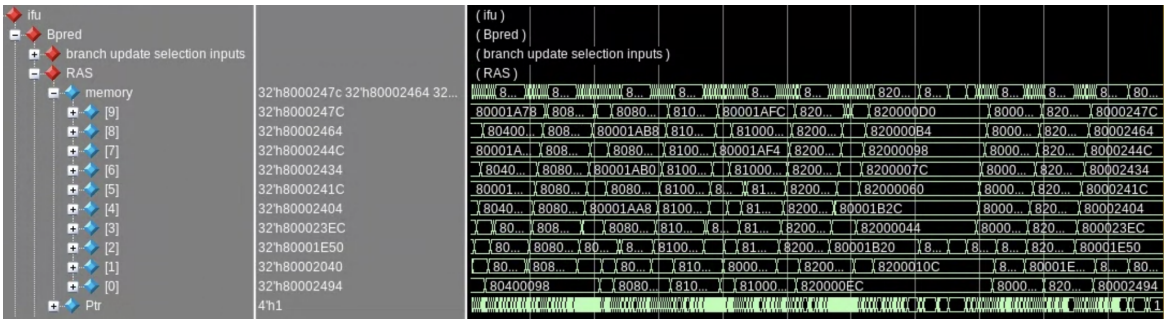


Figure 2.7: RAS (Test 3/4) Waveform

2.3.2 Scaling

Test 3 is a collection of **targeted behavioral probes** rather than a systematic march, therefore it is harder to scale to a bigger RAS. There is no single parameter to increase: chain depth, overflow count, TDF pair count and pointer bit coverage must be updated independently, a consequence of the stress-based rather than march-based idea behind the test.
Phases 1 and 2 (the deep call chains) must be exactly as deep as the RAS has entries (e.g. if you increase to 16 entries, you need 16-level chains). Phase 3's overflow must use `RAS_SIZE + 1` pushes to guarantee exactly one wrap, while Phase 6 (TDF Pairs) has `RAS_SIZE` as number of pairs.

Pointer Width

10-entry RAS uses a 4-bit pointer ($\lceil \log_2(10) \rceil = 4$), but scaling to 32 entries requires 5 bits. Each additional pointer bit introduces new stuck-at and transition fault targets that the current wraparound phase does not cover. Phase 3 would therefore need multiple overflow sequences, one wrapping at exactly `RAS_SIZE`, and ideally one exercising each bit of the pointer independently.

2.4 Test 4

The `sbst4.S` file implements a **MATS+** March test for the RASPredictor. As mentioned before:

$$\text{MATS}_{RAS} = \{\uparrow (w_0); \downarrow (r_0) \uparrow (w_1); \downarrow (r_1) \uparrow (w_0)\} \tag{2.1}$$

The linker script is changed to force subroutines into specific, distinct memory zones:

- **The w_0 (Write 0) Element:** This is executed by issuing a `call` (`jal`) instruction from a subroutine located in the `.ras_low` memory zone (e.g., `0x80002304`). When the `call` executes,

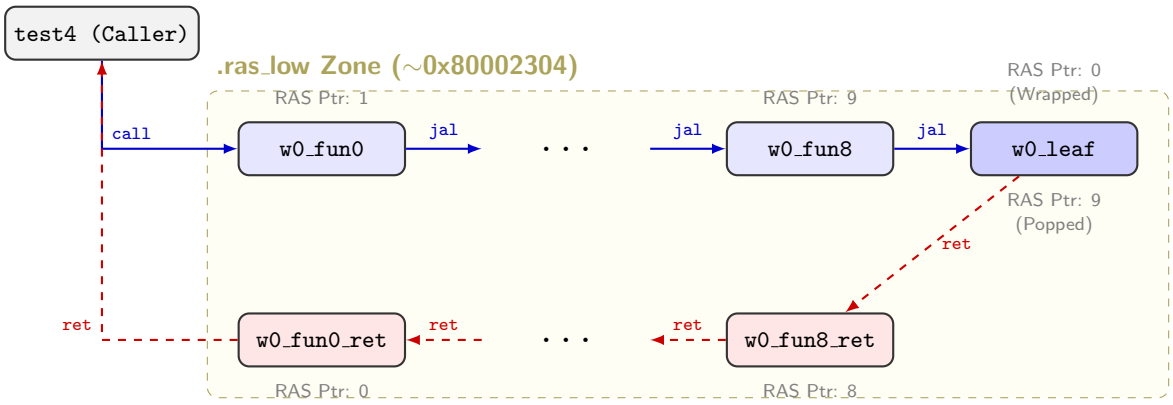
the hardware automatically pushes the sequential Program Counter (PC + 4) into the RAS. Because the PC is in a "low" address space, the binary representation of the address is mostly with 0s.

- **The r_0 (Read 0) Element:** This is executed by issuing a `ret` instruction at the end of the called subroutine. The hardware pops the predicted address from the RAS. If the RAS cell is functioning correctly, it will return the expected low address, and the control flow returns safely. If a Stuck-At-1 corrupted the low address, the processor will mispredict the return, causing a pipeline flush. We detect this by reading the Hardware Performance Monitor (HPM) branch misprediction counter (CSR `0xB04`).
- **The w_1 (Write 1) Element:** To write the opposite pattern, the code jumps to a `call` chain located in the `.ras_high` memory zone (e.g., `0x82001170`). The `call` now pushes a "high" PC value into the RAS, filling the physical SRAM cells with 1s to overwrite the previous 0s.
- **The r_1 (Read 1) Element:** A `ret` instruction is executed to unwind from the `.ras_high` zone. The RAS pops the high address. If a Stuck-At-0 or a Transition Fault occurred, the popped high address will not match the target, causing a measurable misprediction.

2.4.1 Mats+ Flow

The march follows this sequence:

1. **Phase 1 ($\uparrow w_0, \downarrow r_0$):** A chain of nested `call`s descends into the `.ras_low` zone, filling the entire RAS depth with low addresses. The chain then unwinds via nested `ret`s. The HPM reads are checked; any misprediction indicates a Stuck-At-1 (SA1) fault.
2. **Phase 2 ($\uparrow w_1, \downarrow r_1$):** The processor jumps to the `.ras_high` zone and executes a new nested `call` chain, forcing $0 \rightarrow 1$ transitions in all RAS cells. The chain unwinds via `ret`s. Mispredictions indicate a Stuck-At-0 (SA0) or a Rising Transition Fault (TDF-R).
3. **Phase 3 ($\uparrow w_0, \downarrow r_0$):** A secondary nested `call` chain in the `.ras_low` zone is executed to force $1 \rightarrow 0$ transitions.



2.4.2 Algorithm scaling

Mats+ can be scaled by increasing the number of calls/return pair to the size we want, while mantaining the `res_low` and *res_high* memory sections. Increasing the size of the Raspredictor unit requires adding modre nodes like this, nested in the call hierarchy:

Listing 2.3: Generic March element node

```
# ============================================================
# Generic Subroutine Node 'i' in the March Chain
# ============================================================

wX_fun_i:
    # 1. Save the parent's return address to standard memory
    addi sp, sp, -4
    sw   ra, 0(sp)

    # 'call' forces the hardware to push the next sequential
    # instruction (wX_fun_i_ret) into the RAS SRAM.
    call wX_fun_i+1

wX_fun_i_ret:
    # 3. We land here after the child function unwinds.
    # Restore this function's actual return address.
    lw   ra, 0(sp)
    addi sp, sp, 4

    # 'ret' forces the hardware to pop the RAS to predict
    # the jump. If the popped address does not match 'ra',
    # a measurable pipeline flush occurs.
    ret
```

2.4.3 MATS+ Structural Implementation

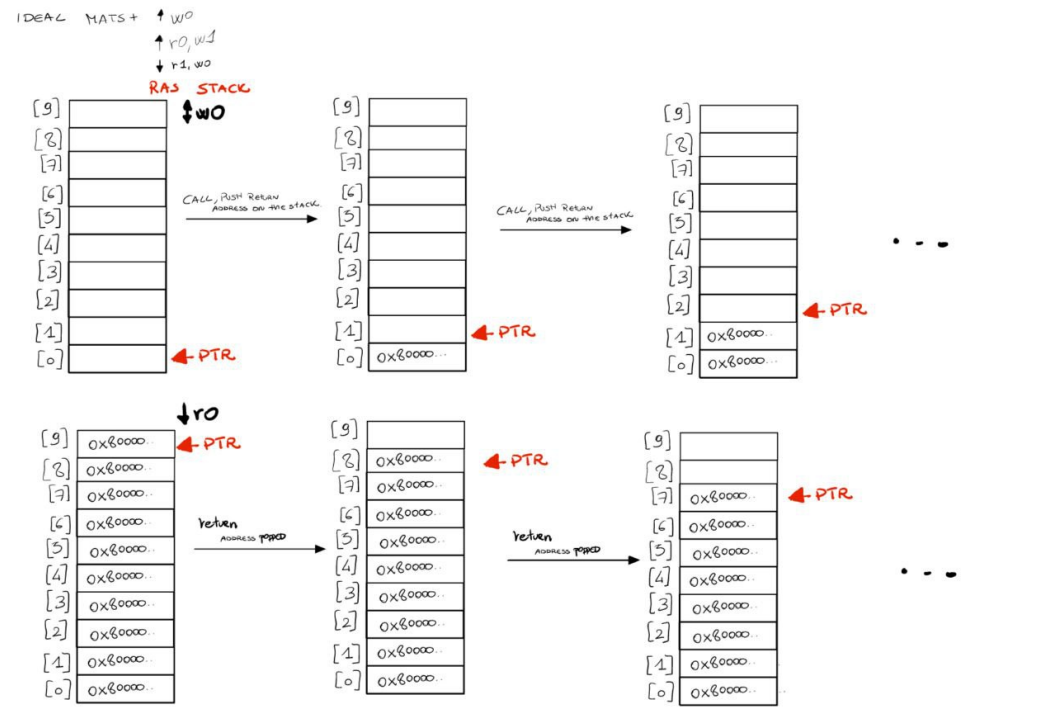


Figure 2.8: Part 1

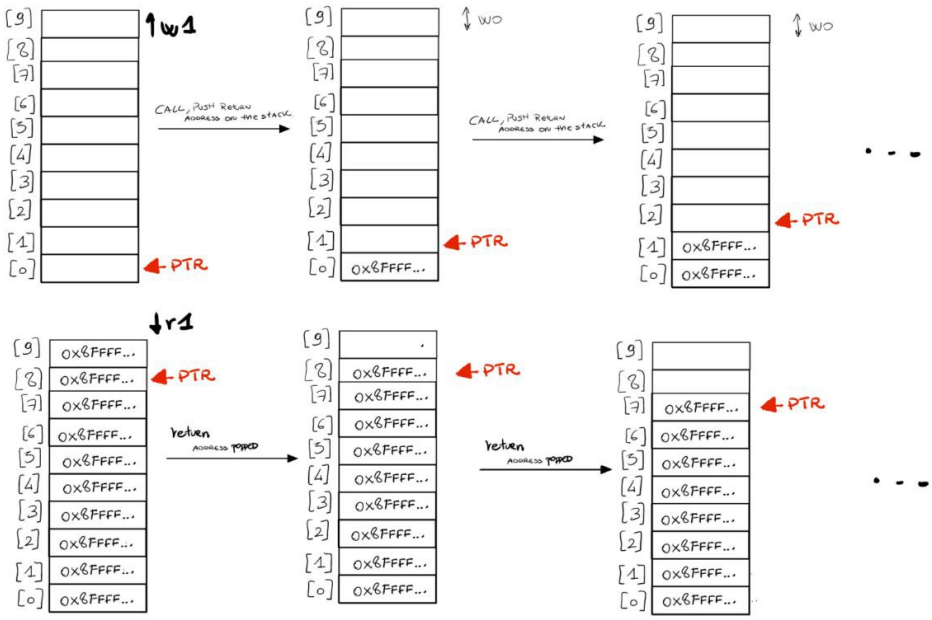


Figure 2.9: Part 2

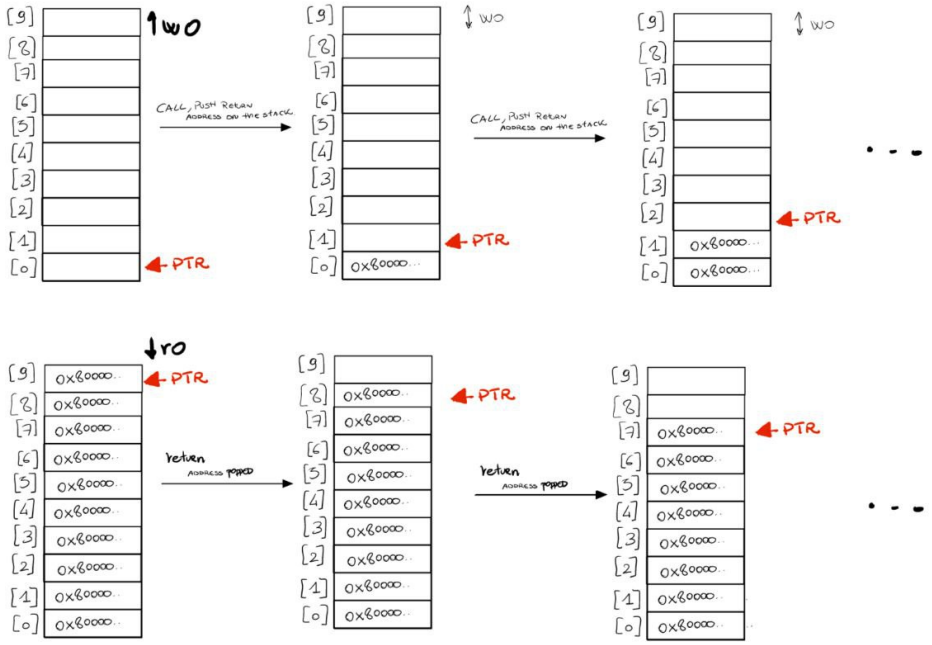


Figure 2.10: Part 3

CHAPTER 3

Fault Coverage

Fault coverage is measured using the **ZOIX fault simulator**, which operates on the gate-level netlist. The flow injects each fault from the fault list individually and compares the resulting simulation against a golden reference VCD. A fault is marked Detected (DD) if it causes an observable difference, Potentially Detected (PD) if a difference is possible but unconfirmed, and Not Detected (ND) if no observable difference is produced.

3.1 Results

3.1.1 Test 1

Test 1 was run in isolation to establish a baseline for BHT coverage. The results show that the march-based approach is effective for the `DirPredictor` logic: 89.28% of faults are detected with an additional 5.70% potentially detected, giving a fault coverage of 38.37% overall (prime faults). The 91.53% detection rate on the `BHT` confirms that the counter state transitions are being exercised correctly.

Listing 3.1: ZOIX sbst1.S coverage

```
Total     ND            DD            PD            UU          UT       UB  Scope
-----  ------------  ------------  ------------  ----------  -------  --  -----
  #SAF
  877    44 (5.02%)  783(89.28%)  50 (5.70%)  16 (1.82%)  1 (0.11%) 0  ---DirPredictor
  555     9 (1.62%)  508(91.53%)  38 (6.85%)  16 (2.88%)  1 (0.18%) 0  ----BHT
  ...

  #TDF
  876    55 (6.28%)  719(82.08%)  102(11.64%) 16 (1.83%)  2 (0.23%) 0  ---DirPredictor
  554    17 (3.07%)  447(80.69%)  90 (16.25%) 16 (2.89%)  2 (0.36%) 0  ----BHT
  ...
                           Prime     Total
# Overall  Test Coverage:  41.85%    41.63%
# Overall Fault Coverage:  38.37%    38.79%
```

3.1.2 Test 2

Adding Test 2 extends coverage to the BTB and marginally improves BHT coverage through additional branch activity. The BTB (`TargetPredictor`) reaches 69.13% DD and 7.45% PD for SAF, reflecting the stress-based/March Alg. approach, though the 23.42% ND rate indicates that systematic march-based coverage of individual BTB entries remains incomplete. The total fault coverage rises to 49.02% on prime faults and 49.47% on total faults.

Listing 3.2: ZOIX sbst1.S + sbst2.S coverage

```
Total     ND            DD            PD            UU          UT       UB  Scope
-----  ------------  ------------  ------------  ----------  -------  --  -----
  #SAF
  877    39 (4.45%)  788(89.85%)  50 (5.70%)  16 (1.82%)  1 (0.11%) 0  ---DirPredictor
  555     7 (1.26%)  510(91.89%)  38 (6.85%)  16 (2.88%)  1 (0.18%) 0  ----BHT
```

```
10465 2451(23.42%) 7234(69.13%) 780 (7.45%) 1408(13.45%) 5 (0.05%) 0  ---TargetPredictor(BTB)
  ...

  #TDF
  876   42 (4.79%)  732(83.56%) 102(11.64%)  16 (1.83%)  2 (0.23%) 0  ---DirPredictor
  554   7  (1.26%)  457(82.49%)  90(16.25%)  16 (2.89%)  2 (0.36%) 0  ----BHT
10460 2835(27.10%) 6773(64.75%) 852(8.15%)  1408(13.46%) 10(0.10%) 0  ---TargetPredictor(BTB)
  ...
                           Prime    Total
# Overall  Test Coverage:  53.47%   53.10%
# Overall Fault Coverage:  49.02%   49.47%
```

3.1.3 Test 3

Adding Test 3 brings the RAS into coverage and produces the largest improvements. The `RASPredictor` reaches 70.70% DD and 7.14% PD for SAF, and 63.13% DD for TDF: the lower TDF result reflects the difficulty of exercising transition faults on the RAS pointer increment path, which the mismatch and wraparound phases only address partially. Coverage on the BHT and BTB also improves slightly as the deep call chains generate additional branch activity across those two. The total fault coverage reaches 66.12% on prime faults and 67.93% on total faults.

Listing 3.3: sbst1.S + sbst2.S + sbst3.S coverage

```
Total    ND           DD           PD           UU          UT      UB  Scope
-----  -----------  -----------  -----------  ----------  -------  --  -----
  #SAF
  877   36 (4.10%)  791(90.19%)  50 (5.70%)   16 (1.82%) 1 (0.11%) 0  ---DirPredictor
  555    4 (0.72%)  513(92.43%)  38 (6.85%)   16 (2.88%) 1 (0.18%) 0  ----BHT
10465 2254(21.54%) 7431(71.01%)  780(7.45%) 1408(13.45%) 5 (0.05%) 0  ---TargetPredictor(BTB)
 9138 2025(22.16%) 6461(70.70%)  652(7.14%)  200 (2.19%) 8 (0.09%) 8  ---RASPredictor
  ...

  #TDF
  876   39 (4.45%)  739(84.36%)  98 (11.19%)  16 (1.83%) 2 (0.23%) 0  ---DirPredictor
  554    4 (0.72%)  464(83.75%)  86 (15.52%)  16 (2.89%) 2 (0.36%) 0  ----BHT
10460 2634(25.18%) 6982(66.75%)  844(8.07%) 1408(13.46%) 10(0.10%) 0  ---TargetPredictor(BTB)
 9130 2718(29.77%) 5764(63.13%)  648(7.10%)  200(2.19%)  16(0.18%) 8  ---RASPredictor
  ...
                           Prime    Total
# Overall  Test Coverage:  72.12%   72.91%
# Overall Fault Coverage:  66.12%   67.93%
```

3.1.4 Test 4 (Final Coverage)

Adding Test 4 further refines coverage of the RASPredictor through a MATS+ march, producing modest improvements across all components. The RASPredictor reaches 70.96% DD and 7.14% PD for SAF, and 63.64% DD for TDF, slightly improving on Test 3's results by ensuring that all RAS entries are exercised through complete 0→1 and 1→0 transitions via the low and high memory zones. The incremental gains over Test 3 are limited by the same architectural address constraints that affect the RAS: bits 0, 1, 31, and 26–30 remain structurally untestable in this memory map configuration. The final overall fault coverage reaches 66.81% on prime faults and 68.56% on total faults.

Listing 3.4: ZOIX Final coverage

```
Total    ND           DD           PD           UU          UT      UB  Scope
-----  -----------  -----------  -----------  ----------  -------  --  -----
  #SAF
  877   37 (4.22%)  790(90.08%)  50 (5.70%)   16 (1.82%) 1 (0.11%) 0  ---DirPredictor
  555    5 (0.90%)  512(92.25%)  38 (6.85%)   16 (2.88%) 1 (0.18%) 0  ----BHT
10465 2174(20.77%) 7511(71.77%)  780(7.45%) 1408(13.45%) 5 (0.05%) 0  ---TargetPredictor(BTB)
 9138 2015(22.05%) 6471(70.81%)  652(7.14%)  200 (2.19%) 8 (0.09%) 8  ---RASPredictor
  ...
```

```
  #TDF
  876   40 (4.57%)  730(83.33%)  106(12.10%)  16 (1.83%) 2 (0.23%) 0  ---DirPredictor
  554    5 (0.90%)  455(82.13%)  94 (16.97%)  16 (2.89%) 2 (0.36%) 0  ----BHT
10460 2381(22.76%) 7227(69.09%)  852(8.15%) 1408(13.46%) 10(0.10%) 0  ---TargetPredictor(BTB)
 9130 2685(29.41%) 5797(63.49%)  648(7.10%)  200(2.19%)  16(0.18%) 8  ---RASPredictor
  ...
                           Prime    Total
# Overall  Test Coverage:  72.87%   73.59%
# Overall Fault Coverage:  66.81%   68.56%
```

3.1.5 Test Application Time (TAT)

The overall Test Application Time was measured at **492.7s** CPU time and 501.8s elapsed time.

Listing 3.5: ZOIX Test Application Time (TAT)

```
CPU Time: 492.7s ; Elapsed Time: 501.8s ; Data Size: 271.6MB
```

While the achieved fault coverage is moderate, our idea was to balance completeness with efficiency. A more exhaustive test could improve coverage, but would do so at the cost of a longer simulation time; therefore, we consider the current results to be a good compromise between FC and TAT.

Listing 3.6: ZOIX Simualtion time

```
Info:    VCD stimulus completed with 0 mismatches.

Info:    Last simulation time was at 223150000 ps
Info:    Logic Events: 26616594
Info:    CPU Time: 16.1s ; Elapsed Time: 16.2s ; Data Size: 146.7MB
Info:    Simulation Completed Sat Mar  7 19:17:49 2026
```

CHAPTER 4

Conclusions and Future Work

4.1 Analysis of our Results

The march-based approach in Test 1 proves **effective** for the BHT: by driving all 8 entries through every counter state transition in both directions, Test 1 achieves 89.28% DD and 91.53% DD on the `BHT` specifically, confirming that the address decoding logic and the saturating counter update path are well exercised.
Test 2 was run in sequence with Test 1: the moderate fault and test coverage of the BTB can be explained by a constraint of the SBST approach applied to the `TargetPredictor`. Unlike the BHT, where the stored values are 2-bit saturating counters that can be freely written to any of four states, the BTB stores **target addresses**, real instruction addresses in the processor's memory map. In our configuration, executable memory is constrained to a specific range, which means that the upper bits of every address stored in the BTB are always `10000000 0...`, and can never be toggled.

Listing 4.1: Uncore RAM base address and range definitions, from deriv

```
localparam logic UNCORE_RAM_SUPPORTED = 1;
localparam logic [63:0] UNCORE_RAM_BASE  = 64'h80000000;
localparam logic [63:0] UNCORE_RAM_RANGE = 64'h07FFFFFF;
```

A march element that requires writing both 0 and 1 to a given bit position is simply not achievable for those bits: no valid branch target can be constructed with those bits cleared. This leaves a significant fraction of the BTB data storage permanently unexercisable through pure SBST and the 23% ND rate on the `TargetPredictor` is a direct consequence.

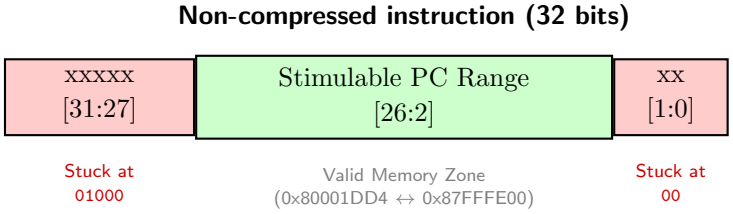


Figure 4.1: Architectural address constraints within the BTB entry. The red sections highlight bits that are unstimulable via software due to hardcoded linker address constraints and strict 32-bit instruction alignment.

The same constraint applies to the RAS in Test 3, though the situation is slightly better. The RAS stores return addresses (the value of PC+4 at the point of each `jal`) which are equally constrained to the `0x80000000`–`0x82000000` range. Phase 8 of Test 3 partially mitigates this by placing deep chains at addresses that individually toggle bits 16 to 25, ensuring that each of those bits sees both a 0 and a 1 written across the 10 RAS entries. However, bits 0 and 1 are always 0 due to instruction alignment, bit 31 is always 1 (RISC-V memory base), and bits 26 to 30 can never be set given the memory map constraints. This leaves roughly 8 bits of the 32-bit RAS data width that are structurally

untestable through pure SBST in this environment, regardless of how many chains are added. Test 4 yielded similar results, with the MATS+ march producing only marginal improvements over Test 3, confirming that the remaining undetected faults are not addressable through additional software sequences but are instead a structural consequence of the memory map constraints.

4.2 Extensions of our Implementation

A first look can be given at the used configuration, since that is where the most immediate extensions can be made.

4.2.1 Architectural Scaling/Changes

The current test suite is tightly coupled to a specific BPU configuration: 8-entry BHT/BTB tables (`BPRED_SIZE = BTB_SIZE = 3`) and a 10-entry RAS. While this was sufficient for the scope of this project, a natural extension would be to validate that the methodology scales correctly to **larger configurations**. For the BHT and BTB, increasing the table size changes the indexing hash and the number of entries that must be independently targeted, which impacts the linker script placement and the number of march iterations required. A systematic study could characterize how fault coverage evolves as table size grows, and whether the current march strategies remain sufficient or need to be augmented. For the RAS, larger stacks introduce more pointer states and wraparound conditions, requiring additional overflow phases in Test 3 to guarantee full coverage of the pointer logic.

Other Branch Prediction Types

Beyond scaling the existing configuration, the test suite currently targets only the `BP_TWOBIT` bimodal predictor, which was chosen for its simple and regular structure. More sophisticated predictor types, such as **GShare**, introduce a global history register that is XORed with the PC to index the BHT, adding a new element that interacts with the counter array. Testing GShare would require exercising not only the counter transitions but also the correctness of the history update logic, since a stuck-at fault in the history register can corrupt predictions without ever asserting a misprediction signal on a single isolated branch.

4.2.2 Automation of the BHT Test

A limitation of the current implementation is that the linker script addresses for BHT targeting were computed manually using a standalone **Python script**, and are therefore hardcoded to the specific derivative configuration used in this project. Any change to `BPRED_SIZE`, `BTB_SIZE`, or the `UNCORE_RAM_RANGE` would invalidate all placement directives and require the entire address computation to be repeated by hand. An important contribution would be to integrate this computation directly into the system, so that the Python script is invoked automatically as part of the make flow, reads the active derivative parameters and regenerates the linker script sections accordingly.

CHAPTER 5

How to Run

The complete workflow is automated by a single script, `run_workflow.sh`, located in the root of the delivery archive. It performs all steps in sequence and exits immediately on any error.

Listing 5.1: One-click workflow execution

```
./run_workflow.sh
```

The script executes the following steps:

1. **Environment setup**: sources `eda_tools_setup` to initialize all required tool paths and environment variables.
2. **Compile the SBST**: runs `make` in `examples/asm/sbst/`, producing `sbst.elf` from the assembly/C sources (`crt0.S`, `main.c`, `sbst1.S`, `sbst2.S`, `sbst3.S`) and the linker script (`link/link_sbst.ld`).
3. **Gate-level simulation**: runs `wsim` with the compiled ELF and the `syn_polito_rv32e_bpu` derivative, generating a VCD of the gate-level netlist using Questa.
4. **Fault simulation**: runs `zoix_cvw.sh` from the `zoix/` directory, injecting faults defined in `fault_gen_cvw.sff` and comparing against the golden VCD using the strobe configuration in `strobe_cvw.sv`.

Prerequisites: the gate-level netlist for the `syn_polito_rv32e_bpu` derivative must already exist before running the script, as synthesis is not included in the automated flow. The netlist is included in the delivery archive and does not need to be regenerated.

APPENDIX A

Code Snippets

Listing A.1: Wally Derivative Configuration List

```
deriv   syn_polito_rv32e_bpu       syn_polito_rv32e
E_SUPPORTED              0
BPRED_SUPPORTED          1
BPRED_TYPE           `BP_TWOBIT
BPRED_SIZE               32'd3
BTB_SIZE                 32'd3
ZICNTR_SUPPORTED         1
ZIHPM_SUPPORTED          1
COUNTERS                 12'd32
RAS_SIZE                 32'd10
```

Listing A.2: Linker script section placement for BHT entry targeting

```
.text : { *(. text ) }

  . = 0x80006820;
  .jumptk0 : { KEEP(*(.jumptk0)) }

  . = 0x80006864;
  .jumptk1 : { KEEP(*(.jumptk1)) }

  . = 0x800068A8;
  .jumptk2 : { KEEP(*(.jumptk2)) }

  . = 0x800068EC;
  .jumptk3 : { KEEP(*(.jumptk3)) }

  . = 0x80006930;
  .jumptk4 : { KEEP(*(.jumptk4)) }

  . = 0x80006974;
  .jumptk5 : { KEEP(*(.jumptk5)) }

  . = 0x800069B8;
  .jumptk6 : { KEEP(*(.jumptk6)) }

  . = 0x800069FC;
  .jumptk7 : { KEEP(*(.jumptk7)) }
```

Listing A.3: Linker script configuration (Test 2) for low memory (Logical 0) sections.

```
  . = ALIGN(64);
  .w0_entry0 : { KEEP(*(.w0_entry0)) }

  . = ALIGN(64);
  .w0d_entry0 : { KEEP(*(.w0d_entry0)) }
  .w0d_entry2 : ALIGN(8) { KEEP(*(.w0d_entry2)) }
  .w0d_entry4 : ALIGN(8) { KEEP(*(.w0d_entry4)) }
```

```
  .w0d_entry6 : ALIGN(8) { KEEP(*(.w0d_entry6)) }
  . += 4;
  .w0d_entry1 : { KEEP(*(.w0d_entry1)) }
  .w0d_entry3 : { KEEP(*(.w0d_entry3)) }
  .w0d_entry5 : { KEEP(*(.w0d_entry5)) }
  .w0d_entry7 : { KEEP(*(.w0d_entry7)) }
```

Listing A.4: Linker script configuration (Test 2) for high memory (Logical 1) sections.

```
  . = 0x87FFFE00;
  .w1_entry0 : { KEEP(*(.w1_entry0)) }
  .w1_entry1 : ALIGN(8) { KEEP(*(.w1_entry1)) }
  .w1_entry2 : ALIGN(8) { KEEP(*(.w1_entry2)) }
  .w1_entry3 : ALIGN(8) { KEEP(*(.w1_entry3)) }
  .w1_entry4 : ALIGN(8) { KEEP(*(.w1_entry4)) }
  .w1_entry5 : ALIGN(8) { KEEP(*(.w1_entry5)) }
  .w1_entry6 : ALIGN(8) { KEEP(*(.w1_entry6)) }
  .w1_entry7 : ALIGN(8) { KEEP(*(.w1_entry7)) }
  .w1_entry8 : ALIGN(8) { KEEP(*(.w1_entry8)) }
  .w1_entry9 : ALIGN(8) { KEEP(*(.w1_entry9)) }
```

Listing A.5: Linker script section placement (Test 3) for RAS targeting

```
/* PHASE 1 */
  . = 0x80008000;
  .ras_walk_t3 : { KEEP(*(.ras_walk_t3)) }
/* PHASE 2 */
  . = 0x80040000;
  .ras_fill_t3 : { KEEP(*(.ras_fill_t3)) }
/* PHASE 6 */
  .p6a0 : { KEEP(*(.p6a0)) }
  .p6b0 : { KEEP(*(.p6b0)) }
  .p6a1 : { KEEP(*(.p6a1)) }
  .p6b1 : { KEEP(*(.p6b1)) }
  .p6a2 : { KEEP(*(.p6a2)) }
  .p6b2 : { KEEP(*(.p6b2)) }
  .p6a3 : { KEEP(*(.p6a3)) }
  .p6b3 : { KEEP(*(.p6b3)) }
  .p6a4 : { KEEP(*(.p6a4)) }
  .p6b4 : { KEEP(*(.p6b4)) }
  .p6a5 : { KEEP(*(.p6a5)) }
  .p6b5 : { KEEP(*(.p6b5)) }
  .p6a6 : { KEEP(*(.p6a6)) }
  .p6b6 : { KEEP(*(.p6b6)) }
  .p6a7 : { KEEP(*(.p6a7)) }
  .p6b7 : { KEEP(*(.p6b7)) }
  .p6a8 : { KEEP(*(.p6a8)) }
  .p6b8 : { KEEP(*(.p6b8)) }
  .p6a9 : { KEEP(*(.p6a9)) }
  .p6b9 : { KEEP(*(.p6b9)) }
/* PHASE 3 */
  .p7_1  : { KEEP(*(.p7_1))  }
  .p7_2  : { KEEP(*(.p7_2))  }
  .p7_3  : { KEEP(*(.p7_3))  }
  .p7_4  : { KEEP(*(.p7_4))  }
  .p7_5  : { KEEP(*(.p7_5))  }
  .p7_6  : { KEEP(*(.p7_6))  }
  .p7_7  : { KEEP(*(.p7_7))  }
  .p7_8  : { KEEP(*(.p7_8))  }
  .p7_9  : { KEEP(*(.p7_9))  }
  .p7_10 : { KEEP(*(.p7_10)) }
  .p7_11 : { KEEP(*(.p7_11)) }
```

```
/* PHASE 7 */
  . = 0x80011000;
  .hi_b16 : { KEEP(*(.hi_b16)) }
  .text_libs : { *(.text.*) }
  . = 0x80022000;
  .hi_b17 : { KEEP(*(.hi_b17)) }
  . = 0x80070000;
  .hi_b18 : { KEEP(*(.hi_b18)) }
  . = 0x80080000;
  .hi_b19 : { KEEP(*(.hi_b19)) }
  . = 0x80100000;
  .hi_b20 : { KEEP(*(.hi_b20)) }
  . = 0x80200000;
  .hi_b21 : { KEEP(*(.hi_b21)) }
  . = 0x80400000;
  .hi_b22 : { KEEP(*(.hi_b22)) }
  . = 0x80800000;
  .hi_b23 : { KEEP(*(.hi_b23)) }
  . = 0x81000000;
  .hi_b24 : { KEEP(*(.hi_b24)) }
  . = 0x82000000;
  .hi_b25 : { KEEP(*(.hi_b25)) }
```

Listing A.6: Fault Generation Configuration for BPU

```
FaultGenerate
{
    #SAF
        NA [0,1] {PORT "wallypipelinedcore_gate.ifu.bpred_bpred.**"}
    #TDF
        NA [R,F] {PORT "wallypipelinedcore_gate.ifu.bpred_bpred.**"}
}
```

Listing A.7: Strobe Locations

```
initial begin

    #27;
    forever begin

        $fs_strobe(‘TOPLEVEL );
        #10;
        $display("Strobed at %t", $time);
    end
end
```

Listing A.8: fmsh.log coverage file

```
                                Z01X (TM)

       Version T-2022.06-SP2 for linux64 - Nov 15, 2022

                    Copyright (c) 1999 - 2022 Synopsys, Inc.
   This software and the associated documentation are proprietary to Synopsys,
 Inc. This software may only be used in accordance with the terms and conditions
 of a written license agreement with Synopsys, Inc. All other use, reproduction,
            or distribution of this software is strictly prohibited.

Sat Mar 07 19:17:50 CET 2026
Z01X (TM) Version: T-2022.06-SP2 (64-bit, Nov 15 2022) for Linux 64-bit

26/03/07 19:17:50  Note:    Reading load file:../fault_sim_cvw.fmsh
> set(var=[defines], format=[standard])
26/03/07 19:17:50  Note:    Set defines[format] to "standard".
> set(var=[fdef], method=[fr], fr.fr=[../fault_gen_cvw.sff],abort=[error])
26/03/07 19:17:50  Note:    Set fdef[fr.fr] to "../fault_gen_cvw.sff".
26/03/07 19:17:50  Note:    Set fdef[abort] to "error".
26/03/07 19:17:50  Note:    Set fdef[method] to "fr".
> set(var=[fsim],hyperfault=[0])
26/03/07 19:17:50  Note:    Set fsim[hyperfault] to 0.
> set(var=[defines], progress.enable=[1], progress.limit=[300])
26/03/07 19:17:50  Note:    Set defines[progress.limit] to 300.
26/03/07 19:17:50  Note:    Set defines[progress.enable] to 1.
> set(var=[defines], progress.style=[long])
26/03/07 19:17:50  Note:    Set defines[progress.style] to "long".
> #set(var=[fdef], fstrobe=[../strobe.strobe])
> set(var=[coats],method=[allf])
26/03/07 19:17:50  Note:    Set coats[method] to "allf".
> set(var=[defines],fsim_extra_options=[+vcs+initreg+0])
26/03/07 19:17:50  Warning: Some option changes could affect previous simulation
    results
26/03/07 19:17:50           Existing simulation results will continue to be used
26/03/07 19:17:50  Note:    Set defines[fsim_extra_options] to "+vcs+initreg+0".
26/03/07 19:17:50  Warning: Options will be saved when a design is opened
>
> design()
26/03/07 19:17:52  Note:    Generating design fault universe file
26/03/07 19:17:52  Note:    Job fr2fdef begun
26/03/07 19:17:53  Note:    Job fr2fdef completed: Host: po
26/03/07 19:17:53  Note:    Total: 50382 Prime: 36411 Collapsed: 13971 Untestable:
    3694
26/03/07 19:17:53  Note:    Fault Generation Completed 0 Errors 0 Warnings
26/03/07 19:17:53  Note:    Created design, directory=/home/s338780/cvw_luca/tft-
    assignment/zoix/run_zoix
26/03/07 19:17:53  Note:    Job coats begun
26/03/07 19:17:54  Note:    Job coats completed: Host: po
>
> addtst(test=[cvw], stimtype=[vcd], stim=[${WALLY}/sim/${LOGIC_SIMULATOR}/core_gate.
    vcd],stim.limit.mismatch=[100], dut.stim=[wallypipelinedcore_gate, testbench.dut.
    core_gate], stim_options=[+TESTNAME=cvw,+vcd+clk+clk])
26/03/07 19:17:54  Note:    Adding test: cvw
>
> fsim()
26/03/07 19:17:55  Note:    Performing testability analysis
26/03/07 19:17:55  Note:    Toggle simulation for test: cvw
26/03/07 19:17:55  Note:    Job toggle Test: cvw begun
26/03/07 19:18:14  Note:    Job toggle completed: Host: po Test: cvw
26/03/07 19:18:14  Note:    Testability analysis for Test: cvw
```

```
26/03/07 19:18:14  Note:    Job coats Test: cvw begun
26/03/07 19:18:16  Note:    Job coats completed: Host: po Test: cvw
26/03/07 19:18:19  Note:    Job boots Test: All begun
26/03/07 19:18:20  Note:    Job boots completed: Host: po Test: All
26/03/07 19:18:20  Note:    Updating Testability Report
26/03/07 19:18:20  Note:    Simulating Test: cvw
26/03/07 19:18:20  Note:    Fault simulation on Test: cvw starting
26/03/07 19:18:20  Note:    Generate fault origins for Test: cvw
26/03/07 19:18:20  Note:    Job coats Test: cvw begun
26/03/07 19:18:21  Note:    Job coats completed: Host: po Test: cvw
26/03/07 19:18:21  Note:    Test: cvw selected 36411 out of 36411 remaining faults to
    simulate
26/03/07 19:18:21  Note:    Fault simulation for Test: cvw
26/03/07 19:18:21  Note:    Job fault_sim Test: cvw begun
26/03/07 19:23:25  Status:  [19:23:25] cvw (1 of 1) | Progress:  61.87% | Jobs: 1
    running | Faults simulated: 22528 of 36411 | DD:15404 ND:7094 PD:2078 | Test
    coverage: 45.16% | Time:  5:04
26/03/07 19:23:25           Part: 0 Running | Host: po | Pass: 12 | Simulation Time: 1
     | Last Strobe Time: 0
26/03/07 19:26:06  Note:    Job fault_sim completed: Host: po Test: cvw
26/03/07 19:26:06  Note:    Record simulation results for Test: cvw
26/03/07 19:26:06  Note:    Job boots Test: cvw begun
26/03/07 19:26:07  Note:    Job boots completed: Host: po Test: cvw
26/03/07 19:26:07  Note:    Simulation of test cvw complete
#                               Prime                  Total
# Test: cvw
# Total Faults:                 36411                  50382
# Detected Faults:                  0     0.00%            0     0.00%
# Dropped Detected Faults:      24854    68.26%        35364    70.19%
# Dropped Potential Faults:      3359     9.23%         3424     6.80%
# Potential Faults:                 0     0.00%            0     0.00%
# Not Detected Faults:           8198    22.52%        11594    23.01%
# Simulation Times:
#   Toggle:                   00:00:19
#   Fault Cumulative:         00:07:44
#   Fault Wall Clock:         00:07:46
#
# Cumulative Test Results
# Total Faults:                 36411                  50382
# Detected Faults:                  0     0.00%            0     0.00%
# Dropped Detected Faults:      24854    68.26%        35364    70.19%
# Dropped Potential Faults:      3359     9.23%         3424     6.80%
# Potential Faults:                 0     0.00%            0     0.00%
# Not Detected Faults:           8198    22.52%        11594    23.01%
#
# Untestable Unused:             3250                   3636
# Untestable Tied:                 42                     42
# Untestable Blocked:              14                     16
#
26/03/07 19:26:07  Note:    Simulation Completed.
26/03/07 19:26:07  Note:    Updating Testability Report

# Tests simulated in this session:
#                               Prime                  Total
# Test: cvw
# Total Faults:                 36411                  50382
# Detected Faults:                  0     0.00%            0     0.00%
# Dropped Detected Faults:      24854    68.26%        35364    70.19%
# Dropped Potential Faults:      3359     9.23%         3424     6.80%
# Potential Faults:                 0     0.00%            0     0.00%
# Not Detected Faults:           8198    22.52%        11594    23.01%
# Simulation Times:
```

```
#   Toggle:                   00:00:19
#   Fault Cumulative:         00:07:44
#   Fault Wall Clock:         00:07:46
#
# Cumulative Test Results
# Total Faults:                 36411                   50382
# Detected Faults:                  0     0.00%             0     0.00%
# Dropped Detected Faults:      24854    68.26%         35364    70.19%
# Dropped Potential Faults:      3359     9.23%          3424     6.80%
# Potential Faults:                 0     0.00%             0     0.00%
# Not Detected Faults:           8198    22.52%         11594    23.01%
#
# Untestable Unused:             3250                    3636
# Untestable Tied:                 42                      42
# Untestable Blocked:              14                      16
#
# Test Coverage:                          72.87%                  73.59%
# Fault Coverage:                         66.81%                  68.56%
#
> coverage(group=[detail])
26/03/07 19:26:08  Note:    Running coverage analysis
26/03/07 19:26:08  Note:    Job fault_report completed: Host: po
26/03/07 19:26:08  Note:    Fault report file is sim.rpt
>
> coverage(file=[cvw_coverage.sff])
26/03/07 19:26:08  Note:    Running coverage analysis
26/03/07 19:26:09  Note:    Job fault_report completed: Host: po
26/03/07 19:26:09  Note:    Fault report file is cvw_coverage.sff
> coverage(file=[cvw_coverage_hierachical.sff],hierarchical=[4])
26/03/07 19:26:09  Note:    Running coverage analysis
26/03/07 19:26:11  Note:    Job fault_report completed: Host: po
26/03/07 19:26:11  Note:    Fault report file is cvw_coverage_hierachical.sff
26/03/07 19:26:11  Note:    Script file complete
26/03/07 19:26:11  Note:    Sat Mar 07 19:26:11 CET 2026
CPU Time: 492.7s ; Elapsed Time: 501.8s ; Data Size: 271.6MB
```

Bibliography

[1] E. Sanchez, M. Sonza Reorda, and A. Tonda, "On the Functional Test of Branch Prediction Units based on Branch History Table," Politecnico di Torino, 2011.

[2] A. J. van de Goor, *Testing Semiconductor Memories: Theory and Practice*, John Wiley & Sons, 1991.

[3] M. Psarakis *et al.*, "Microprocessor Software-Based Self-Testing," *IEEE Design & Test of Computers*, vol. 27, no. 3, 2010.

[4] M. Gaudesi, S. Saleem, E. Sanchez, M. Sonza Reorda, and E. Tanowe, "On the In-Field Test of Branch Prediction Units Using the Correlated Predictor Mechanism," 2014.

[5] D. Changdao, M. Graziano, E. Sanchez, M. Sonza Reorda, M. Zamboni, and N. Zhifan, "On the functional test of the BTB logic in pipelined and superscalar processors," *14th IEEE Latin American Test Workshop (LATW)*, 2013, pp. 1–6.

[6] P. Bernardi, L. Ciganda, M. Grosso, E. Sanchez, and M. Sonza Reorda, "A SBST Strategy to Test Microprocessors' Branch Target Buffer," *IEEE 15th International Symposium on Design and Diagnostics of Electronic Circuits and Systems*, 2012, pp. 306–311.